How Computers Work

From a transistor to an iPhone — Chapter 1

Amir Farbin

Session A — From Charge to a Computer

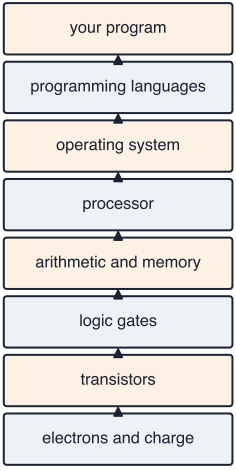
Why this chapter

- Computing is your **medium** — like an artist's materials
- You can use tools without understanding them... to a point
- The more you know about what's underneath, the better you'll use it
- Goal: a gateway — recognize things now, look them up later

Abstraction

- *The quality of dealing with ideas rather than events; considering something independently of its associations, attributes, or concrete accompaniments*
- In practice: solve a problem once $\rightarrow$ seal it behind an input/output barrier $\rightarrow$ never re-derive it $\rightarrow$ build on top
- Group theory: 17 wallpapers $\rightarrow$ crystals $\rightarrow$ forces of nature
- You will live **above and below** abstraction barriers

The ladder we climb today



each layer is an abstraction barrier: solve it once, build on top

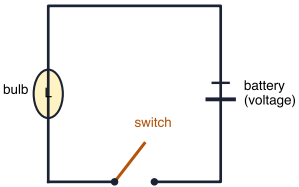
Electrodynamics

- One of the fundamental forces of nature: **electricity & magnetism**
  - What keeps atoms together — responsible for all **chemistry**
  - Foundation of **electronics**
- Electrons have **charge** $\rightarrow$ produce a **field** $\rightarrow$ feel other fields
- Charges move (a **current**) freely in conductors
  - Wires are electron pipes... like water pipes
  - **Voltage** $\sim$ pressure in the pipe

Electrodynamics: circuits

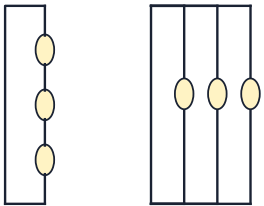
opposite charges attract:
the field between + and –

a circuit: voltage pushes current
through the loop — the switch decides



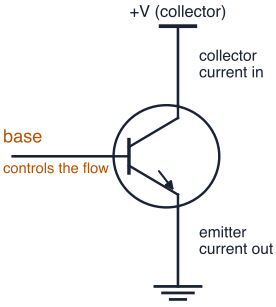
open = no flow = “0” closed = flow = “1”

series vs parallel:
the two ways to combine



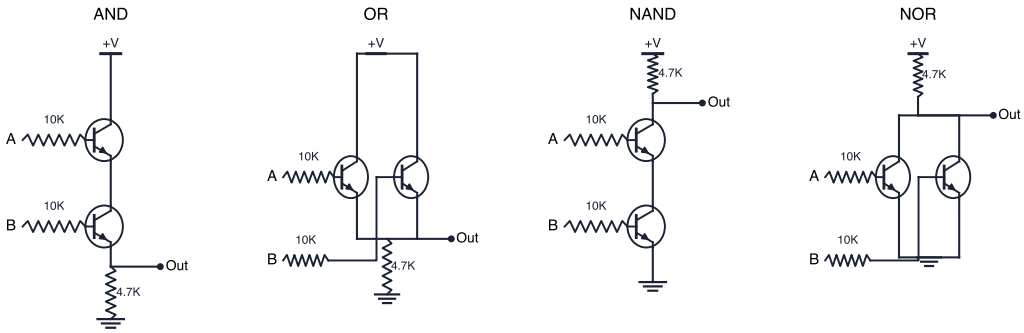
- **Digital electronics** (abstraction!): “0” = no flow · “1” = flow · **clock** = look at a regular rate

The transistor



- **Base**: controls whether current may flow
- **Collector**: current in, when the base allows it
- **Emitter**: current out, to the rest of the circuit
- $\rightarrow$ a **digital switch**

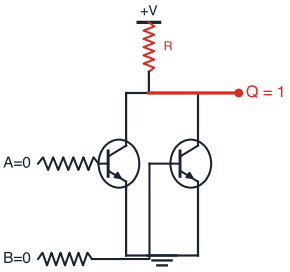
Logic gates — the actual circuits



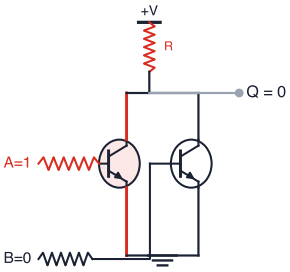
- **Series** transistors → AND-family · **parallel** → OR-family
- Output at top vs bottom → inverted or not

Walking the current: NOR

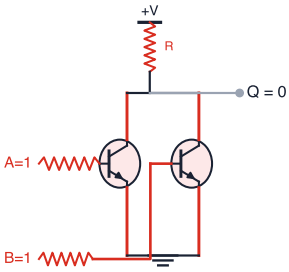
both OFF: charge has nowhere to go but Out: Q = 1



A on: its transistor conducts, charge drains to ground: Q = 0

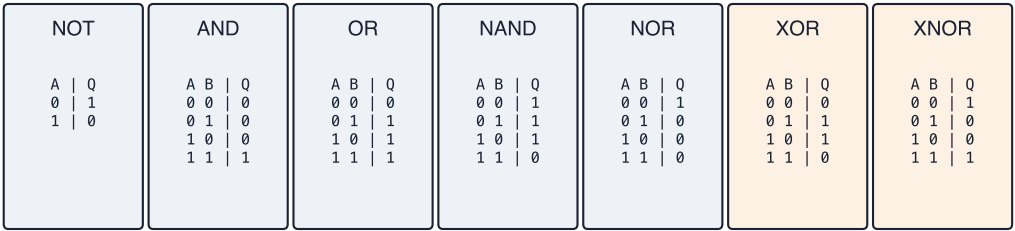


both on: still grounded: Q = 0 (this is NOR)



- The truth table **is** the circuit's behavior — nobody programmed it

Boolean operations — the full set



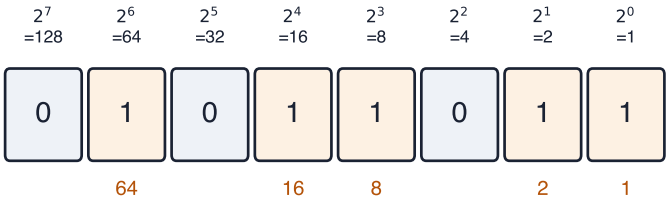
seven operations — every one is a small transistor circuit; from here on, just symbols

- NOT · AND · OR · NAND · NOR · XOR · XNOR
- Each one: a handful of transistors
- **Abstraction**: from now on, just the symbols

Representation

- Language is a representation of **ideas**
  - Writing is a representation of language. Speaking is another. So is a recording.
- In mathematics: a very general relationship expressing **similarities or equivalences** between objects or structures
  - diagrams, number lines, graphs, formulas, physical models...
- **Data representation** — e.g. pixels in an image: the computer sees only numbers encoding color; *we* see the cat
- **Representation learning** — LLMs map concepts into vector spaces where similar things sit near each other (much more later)

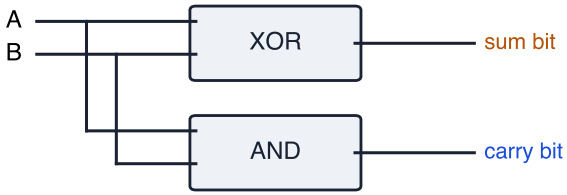
Binary



$64 + 16 + 8 + 2 + 1 = 91$

- Same positional system as decimal — two symbols instead of ten
- 8 bits $\rightarrow$ 256 patterns $\rightarrow$ 0–255, or $-128 \ldots +127$

Arithmetic from logic



A	B	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- sum $=$ XOR $\cdot$ carry $=$ AND
- *Arithmetic is a pattern of logic*

Half adder → full adder → N bits

half adder: 2 bits

```
0+0 = 00
0+1 = 01
1+0 = 01
1+1 = 10
```

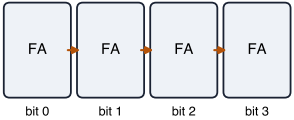
sum = XOR carry = AND

full adder: 2 bits + carry in

```
0+0+0 = 00    1+1+0 = 10
0+1+0 = 01    1+0+1 = 10
1+0+0 = 01    0+1+1 = 10
0+0+1 = 01    1+1+1 = 11
```

two half adders + an OR

N-bit adder: chain the carries



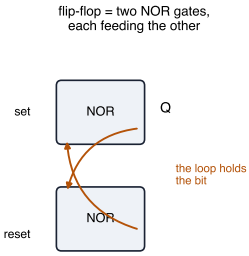
carry ripples left; any width you like

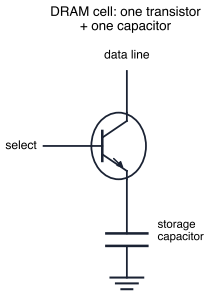
- Full adder: two bits + **carry in** (two half adders + OR)
- Chain them: the carry ripples — any width you like

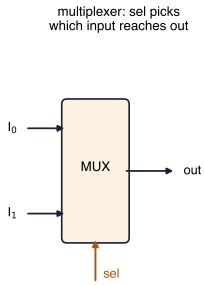
Other circuits

- **Flip-flop**: a circuit that can *store* a value — its output depends on what was written to it earlier
- **Multiplexer**: selects between two inputs I_0 and I_1; outputs I_0 if the command is 0, I_1 if it is 1
  - use it to select different instructions (e.g. $+$ or $-$)
- **Random Access Memory**: 1024-bit multiplexer + 1024 flip-flops = **1 KB**

Memory: the circuits







1024-bit MUX + 1024 flip-flops = 1 KB of RAM

- The flip-flop loop *is* the bit: two NOR gates holding each other in place
- DRAM cell: one transistor guarding one storage capacitor

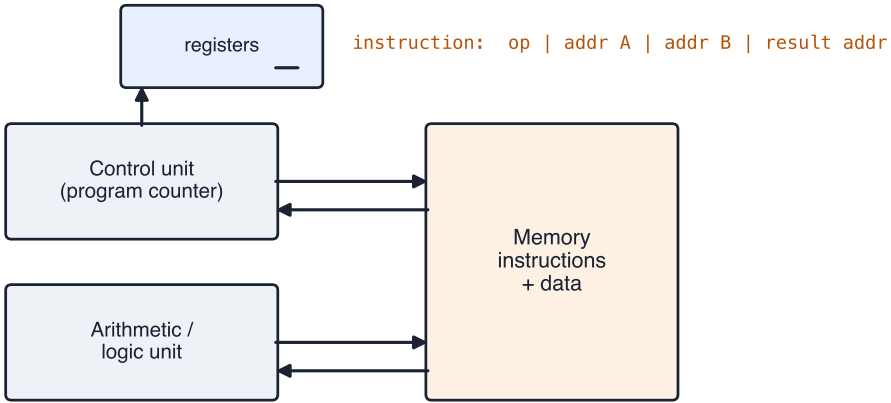
Memory, assembled

- MUX decides *which* flip-flop a bit lands in $\rightarrow$ **random access**
- **RAM**: jump straight to any address
- **Registers**: the handful of values you're working on *right now*

Simple electronic calculator

- Three **registers**, each holding an n-bit number
  - each register is an N-bit flip-flop
- Each **number key** loads that number's binary representation into a register
- An **operation key** ($+$, $-$, ...) takes the two registers as inputs, applies the operation, and puts the result into the third register
- Press **=** $\rightarrow$ the result pushes through
- *You* are the control unit — pressing keys in sequence

Von Neumann architecture



Von Neumann: the parts

- **Memory Unit (MEM)**: holds all the "commands" (**instructions**) and "numbers" (**data**) — together, as bits
- **Control Unit** — with the **program counter**: selects which instruction to execute from MEM; normally just increases by 1 each **clock cycle**
- **Arithmetic block (ALU)**: multiplexers connecting to the different binary operations ($+$, $-$, ...), with inputs and outputs
- **Registers**: hold the input/output of the arithmetic block

Von Neumann: instructions

- Two types: **data instructions** and **control instructions**
- Each **data instruction** contains four things:
  - two addresses saying which two numbers to pick up from MEM
  - one command saying what operation to perform
  - one address saying where to put the result back
- A **control instruction** puts a new address into the **program counter**
  - jump — possibly *conditionally* — and that is control flow
- Clock ticks → fetch, execute, advance → press "=" forever, automatically

Session B — Real Machines

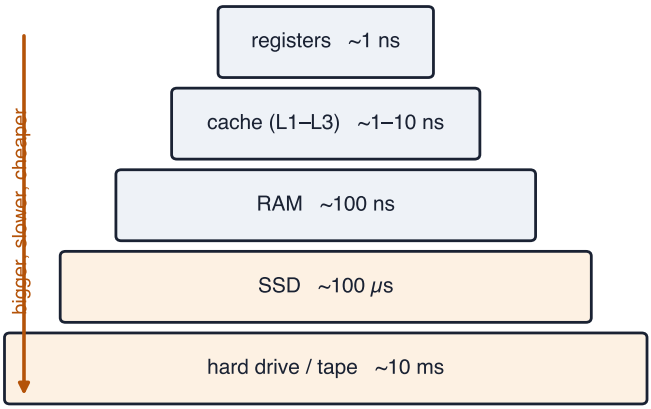
How chips are made

- **Lithography**: print circuits with light, layer by layer
- 386 story: clock speeds climbed for 20 years...
- ...and stopped near 4 GHz: **heat**
- Smaller structures need less power $\rightarrow$ 3 nm, **multicore**
- Real limits today: power in, heat out

Modern processors

- **Variable clock rates** (GHz) — the chip speeds up and slows down
- Biggest constraints: **energy consumption** and **heat dissipation**
- Large amounts of memory (GBs) — usually *not* on the processor
  - new processors (Apple M1/M2/M3): memory integrated with the CPU
- **Cache**: a small part of memory mirrored inside/closer to the processor to accelerate access — multi-level: trade size vs proximity/speed
- **Out-of-order processing** — the chip reorders work to keep busy
- **Many cores**

The memory bottleneck



Apple silicon (why your laptop is fast)

- Memory stacked **on** the processor — close is fast
- CPU + GPU + neural engine share one memory
- Gaming rig: copies CPU-GPU constantly; unified memory: no copies

Floating point

$$\text{value} = (-1)^{sign} \times 1.\text{fraction} \times 2^{exponent - 127}$$

- Scientific notation in binary
- **FLOPS** = cores × clock × ops/cycle
- AI-scale computing is measured in teraflops — and power plants

Motherboard

- **CPU** — central processing unit · **RAM** — random access memory
- **BIOS** — basic input/output system (firmware)
- Off-processor **cache**
- **Expansion bus**: PCI — where you plug in a GPU or other peripherals
- **Chipset** — controls data flow in and out of the CPU
  - *northbridge*: CPU to memory · *southbridge*: CPU to IO
- **IO controllers** — disk (SATA, RAID), peripherals (USB), network (Ethernet, WiFi) — and the **CPU clock**

The whole computer

- **Case** · **power supply** · **fans**
- **Motherboard**, carrying the **CPU** + **heat sink**, **RAM**
- **GPU** — graphics processing unit
- Connectors: USB, Ethernet, ...
- **Storage**: hard drive or solid state drive

Supercomputers

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, **HPE** DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,194.00	1,679.82	22,703
2	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, **Fujitsu** RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
3	**LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, **HPE** EuroHPC/CSC Finland	2,220,288	309.10	428.70	6,016
4	**Leonardo** - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, **Atos** EuroHPC/CINECA Italy	1,824,768	238.70	304.47	7,404
5	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, **IBM** DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148.60	200.79	10,096
6	**Sierra** - IBM Power System AC922, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, **IBM / NVIDIA / Mellanox** DOE/NNSA/LLNL United States	1,572,480	94.64	125.71	7,438
7	**Sunway TaihuLight** - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway, **NRCPC** National Supercomputing Center in Wuxi China	10,649,600	93.01	125.44	15,371
8	**Perlmutter** - HPE Cray EX235n, AMD EPYC 7763 64C 2.45GHz, NVIDIA A100 SXM4 40 GB, Slingshot-10, **HPE** DOE/SC/LBNL/NERSC United States	761,856	70.87	93.75	2,589
9	**Selene** - NVIDIA DGX A100, AMD EPYC 7742 64C 2.25GHz, NVIDIA A100, Mellanox HDR Infiniband, **Nvidia** NVIDIA Corporation United States	555,520	63.46	79.22	2,646

Storage

- RAM forgets at power-off; **storage** persists
- Magnetic: tape $\rightarrow$ floppies $\rightarrow$ hard drives
- **SSD**: silicon, no moving parts, $\sim$10 GB/s vs $<$1 GB/s
- Data centers still spin disks: cheaper per byte
- **Swap**: when RAM runs out, disk pretends — and everything crawls

File systems

- **Sectors** $\rightarrow$ **partitions** $\rightarrow$ **file systems**
- The **file** is an *invention*: named bits
- File table: name $\rightarrow$ blocks $+$ metadata $+$ **permissions**
- Extensions are convention; hierarchy is design (Xerox PARC!)

Booting

- Each stage knows *just enough* to find the next
- Bootloader runs before any file system exists

Session C — Software

The operating system

- Manages hardware; shares CPU, memory, devices among programs
- Programs ask the OS via an **API** — "make me a window"
- **Process** = running program + its resources
- **Preemptive multitasking**: the OS *takes* control back

Unix: born on mainframes

- 1970s Bell Labs, together with **C**
- Shared machines $\rightarrow$ multi-user + multitasking *from birth*
- 1980s open-source: **Linux** and **BSD**
- Unix is a **standard** — with a certification

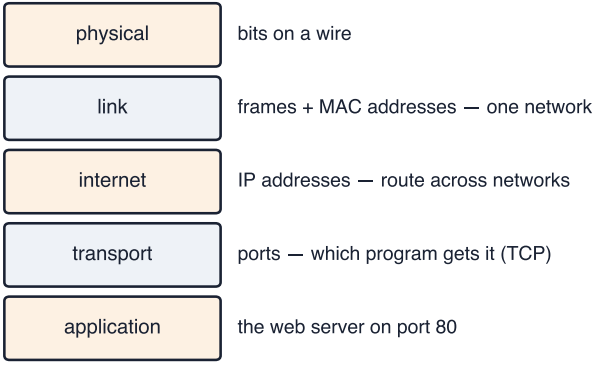
The family tree

- **Distribution** = kernel + GNU tools + packaging (Ubuntu, Debian…)
- macOS = BSD kernel via **NeXT** (the Jobs detour) — certified Unix
- iOS = same system, tuned for a phone
- Windows: CP/M → DOS → Windows — single-user ancestry, retrofitted

The Unix philosophy

- Configuration is **plain text**
- **Everything is a file** — devices, programs, network streams
- Small tools (`ls`, `rm`, `mkdir`), composed in the **shell**
- Chain programs: the output of one is the input of the next

Networks


physical
bits on a wire
link
frames + MAC addresses — one network
internet
IP addresses — route across networks
transport
ports — which program gets it (TCP)
application
the web server on port 80

Networks, continued

- **MAC** address: unique ID on one network
- **IP**: addressing *between* networks, router by router
- **DNS**: names $\rightarrow$ addresses, worldwide phone book
- **Ports**: which program on the machine (web $= 80$)
- Denial of service: drowned by *legitimate* requests

Session D — Mixing Systems & Languages

Two incompatibilities, not one

- Different **API**: a Linux program calls functions Windows doesn't have
- Different **instruction set**: ARM is not x86 machine code
- Which mismatch you have determines the fix

Four fixes

four ways to run one system's software on another

wrapper	translation	virtual machine	container
translate API calls one by one	translate machine instructions	simulate a whole computer	package just the app + its needs
WSL 1, Wine, SteamOS	Rosetta 2, Windows on ARM	WSL 2, cloud servers	Docker, Colab

The ones you'll actually use

- **WSL 2**: a real Linux, in a VM, inside Windows — *two computers*
- The cloud = **hypervisors** booting your **image** on rented hardware
- **Containers**: package the app, share the kernel — Colab does this
- When software won't install: find the container

Programming languages

- Machine code → **assembly** → high-level languages
- **Compiled** (C, C++): translate once, run fast
- **Interpreted** (Python): read on the fly, run conveniently
- Python's trick: orchestrate *compiled* libraries

What makes a language (MIT lesson)

1. **Primitives** — numbers, characters, operations
2. **Means of combination** — expressions, lists
3. **Means of abstraction** — names, functions

- Everything else is convenience
- These are software concepts, not Python facts

Next

- Set up your Unix environment (WSL / macOS)
- Meet the shell properly
- Start building — the fastest way to understand a computer is to make it do something